





# Outlines



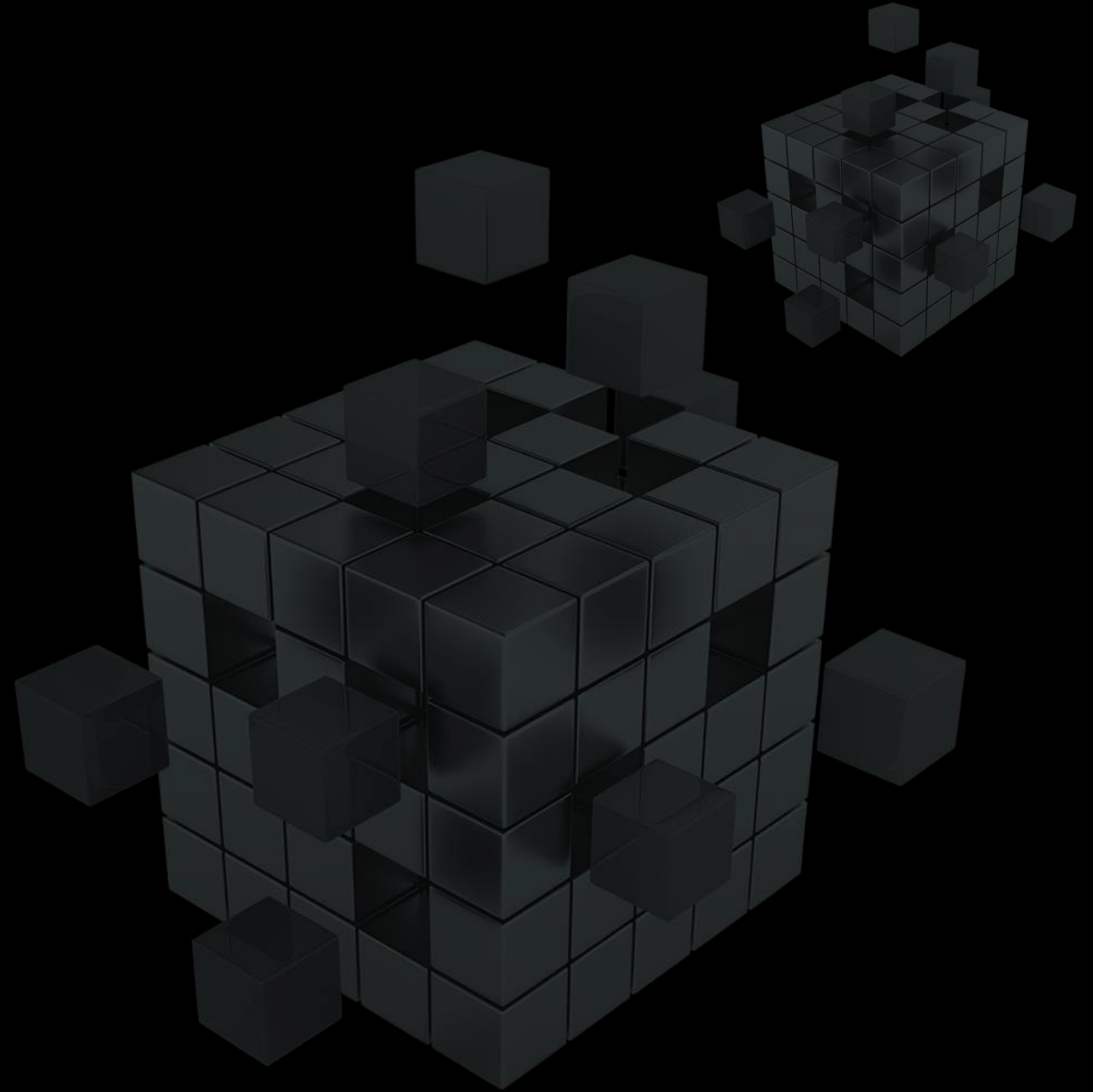
Introduction to Containers



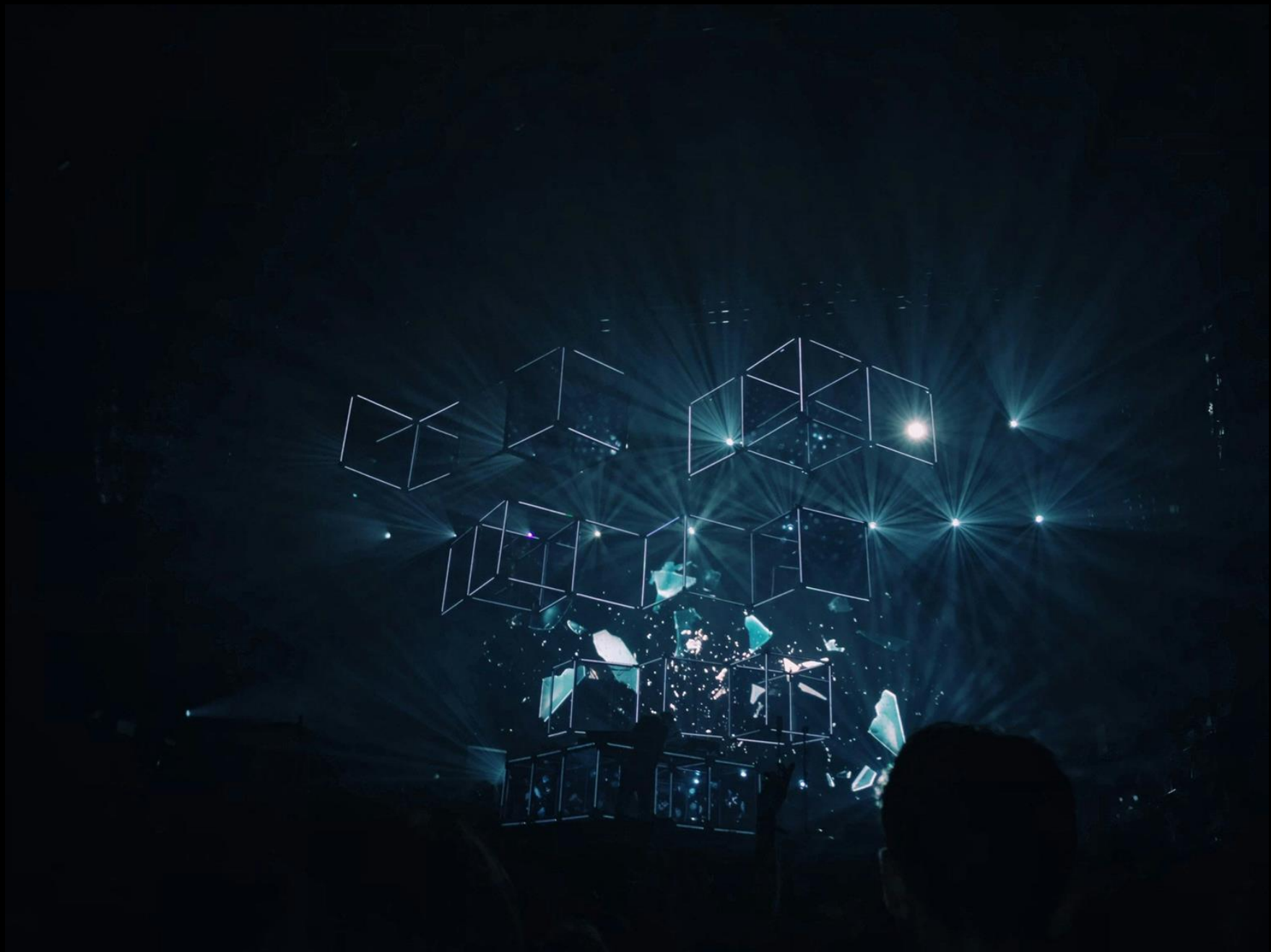
Docker

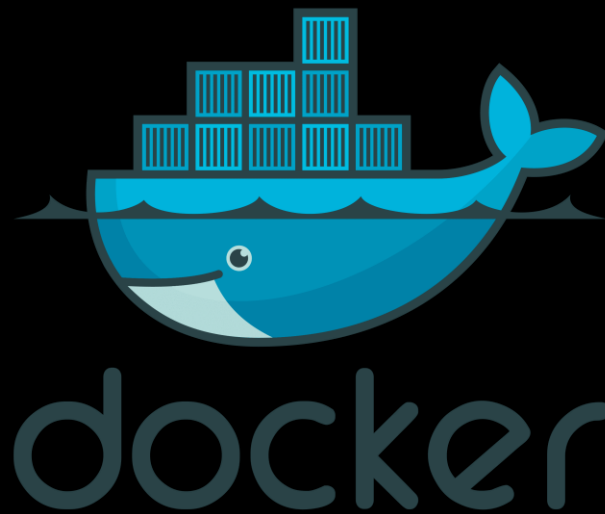


Docker Commands









# Introduction to Containers?

## ➡ We Isolate Services

- To host our applications, we require an infrastructure.
- We use virtual machines (VMs) or cloud computing to set up the infrastructure.
- We isolate our services within the operating system of each VM.
- Due to isolation requirements, we end up deploying multiple VMs or instances.
- These VMs or instances are often overprovisioned.
- This leads to high capital (CapEx) and operational (OpEx) expenses.

# Introduction to Containers?

➡ VM's are **expensive**

- Each virtual machine (VM) includes its own operating system (OS).
- The OS requires maintenance, management and licensing.
- The OS takes time to boot up.
- VMs are portable but they are also heavy and resource-intensive.
- Each VM consumes additional resources just to run its OS.

# Introduction to Containers?

➔ VM's are **expensive**

- Service isolation is important and requires its own operating system.
- High availability is achieved through multiple instances or virtual machines.
- Portability simplifies and streamlines deployment.



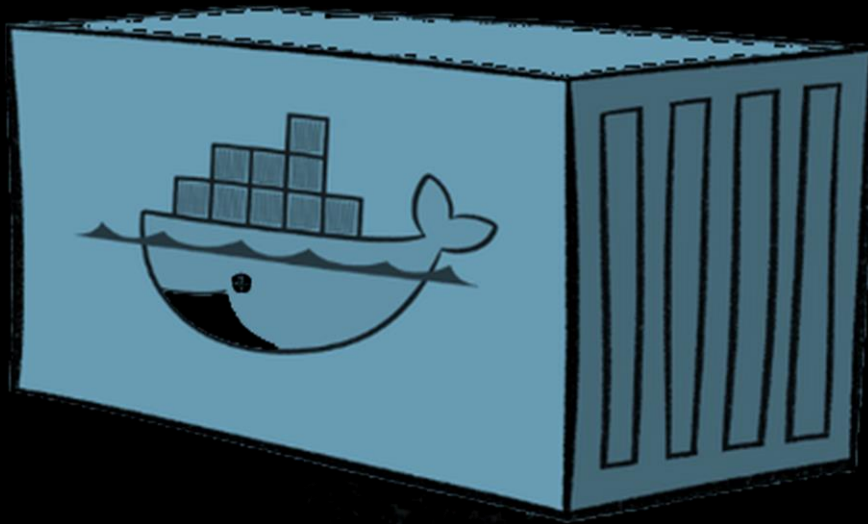
**increases both CapEx and OpEx**

# Introduction to Containers?

➔ Isolation without OS



Imagine multiple services running on the same operating system but fully isolated from each other.



## Container

Process running in a Directory

# Introduction to Containers?

## ➡ The Solution is Containers

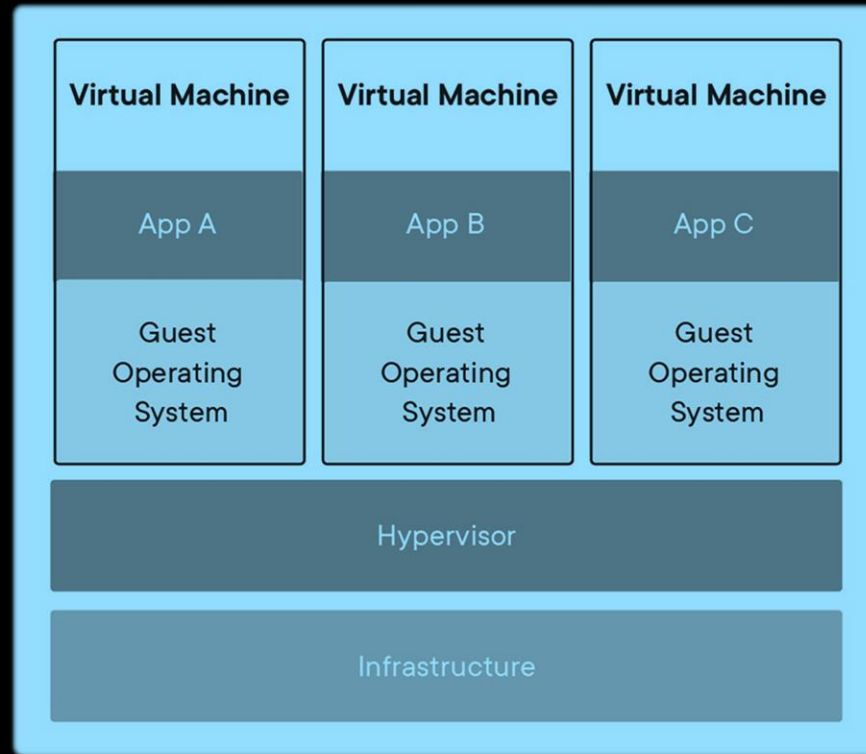
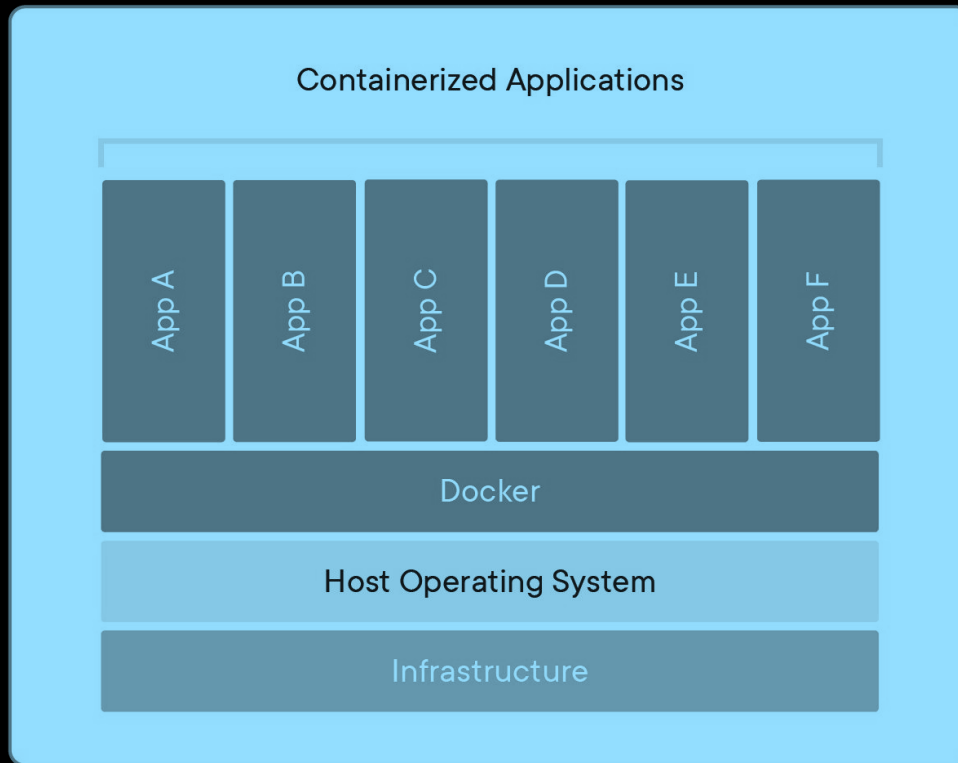
It is an isolated process contained within a directory that has its own namespace, necessary binaries and libraries and a unique IP address for connectivity.

☞ Containers share the host machine's operating system kernel eliminating the need for a separate OS for each application.

☞ The container is a standardized software unit that bundles together application code and all its required dependencies.

# Introduction to Containers?

## ➔ VM vs Container



# Introduction to Containers?

## ➔ VM vs Container

| Aspect              | Container                               | VM                                  |
|---------------------|-----------------------------------------|-------------------------------------|
| Virtualization Type | OS-level virtualization                 | Hardware-level virtualization       |
| Isolation Type      | Isolation not full virtualization       | Fully virtualized environments      |
| OS                  | Do n't require a separate OS            | Each VM needs its own OS            |
| Kernel              | Share the host OS kernel                | Have their own OS kernel            |
| Resource            | Lightweight<br>(host compute resources) | Heavier<br>(require more resources) |
| Performance         | Faster startup and lower overhead       | Slower startup and higher overhead  |

# Docker?

➔ What is **Docker** ?



**Docker is an open-source platform that allows you to build, package and run applications in containers.**



docker

**Docker**

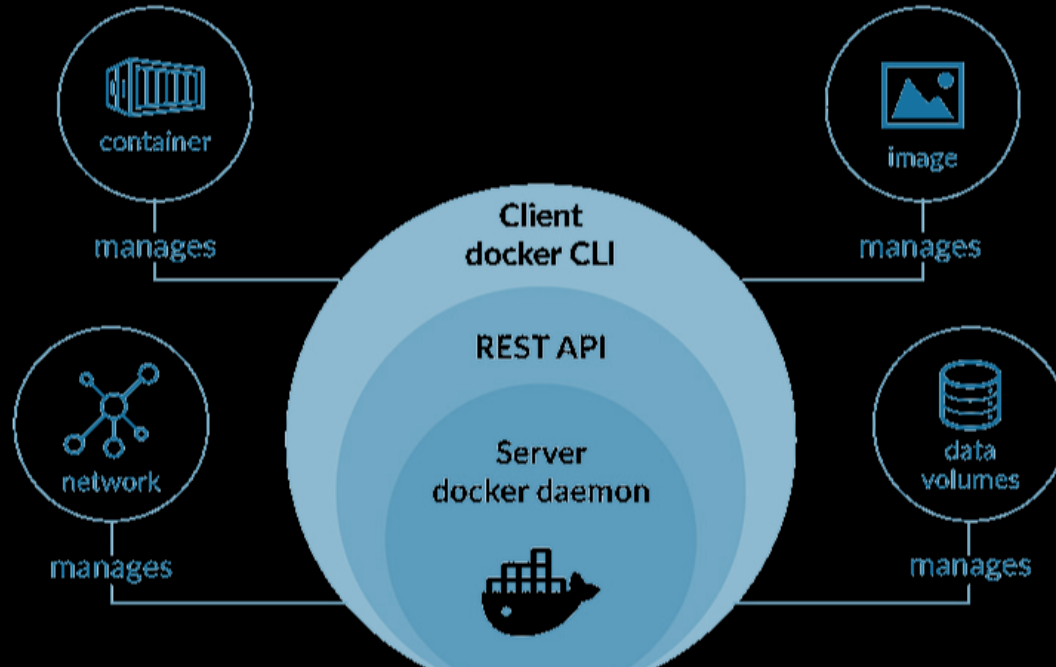
Manages your Containers

# Docker?

## ➔ Docker Engine



**Docker Engine is the core runtime that drives Docker, responsible for building and running containers.**



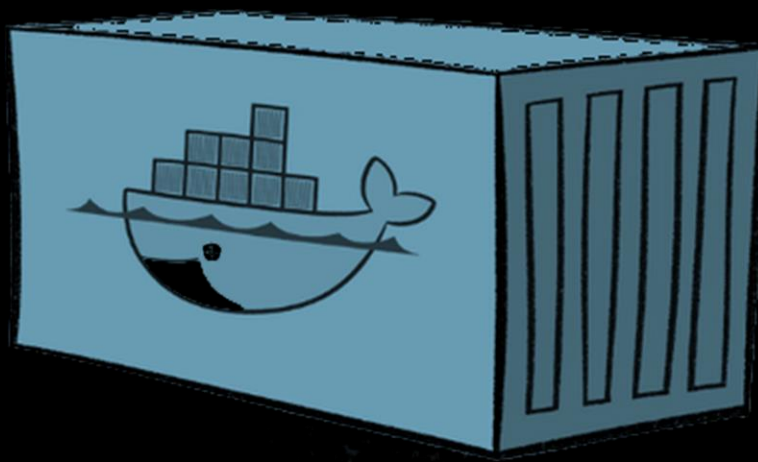
## Docker Engine

**Docker Daemon:** Manages containers and images  
**Docker Client:** Sends commands to the daemon  
**REST API:** Enables communication between client and daemon

# Docker?

## ➡ Docker Container

- ☞ It is a lightweight and isolated environment that runs an application along with all its dependencies.
- ☞ It is a running instance of a Docker image, sharing the host OS kernel but operating as if it's a separate system.



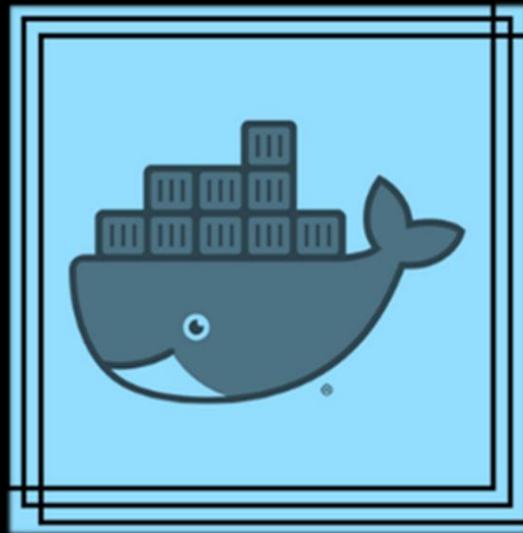
## Docker Container

- Standard:** Portable and consistent across any environment
- Secure:** Provide strong isolation with industry-leading security
- Lightweight:** Share the host OS kernel, improving efficiency and reducing costs

# Docker?

## ➡ Docker Image

- ☞ It is a lightweight, read-only template used to create containers.
- ☞ It contains everything needed to run an application — the code, runtime, libraries, environment variables and configuration files.



## Docker Image

- Blueprint for containers
- Read-only and reusable
- Built using Dockerfile
- Stopped container like VM image
- Consists of multiple layers

# Docker?

## ➔ Docker Registry

- ☞ It is a storage and distribution system for Docker images.
- ☞ It is where Docker images are stored, managed and pulled from when creating containers.



## Docker Registry

Storage for Docker Images

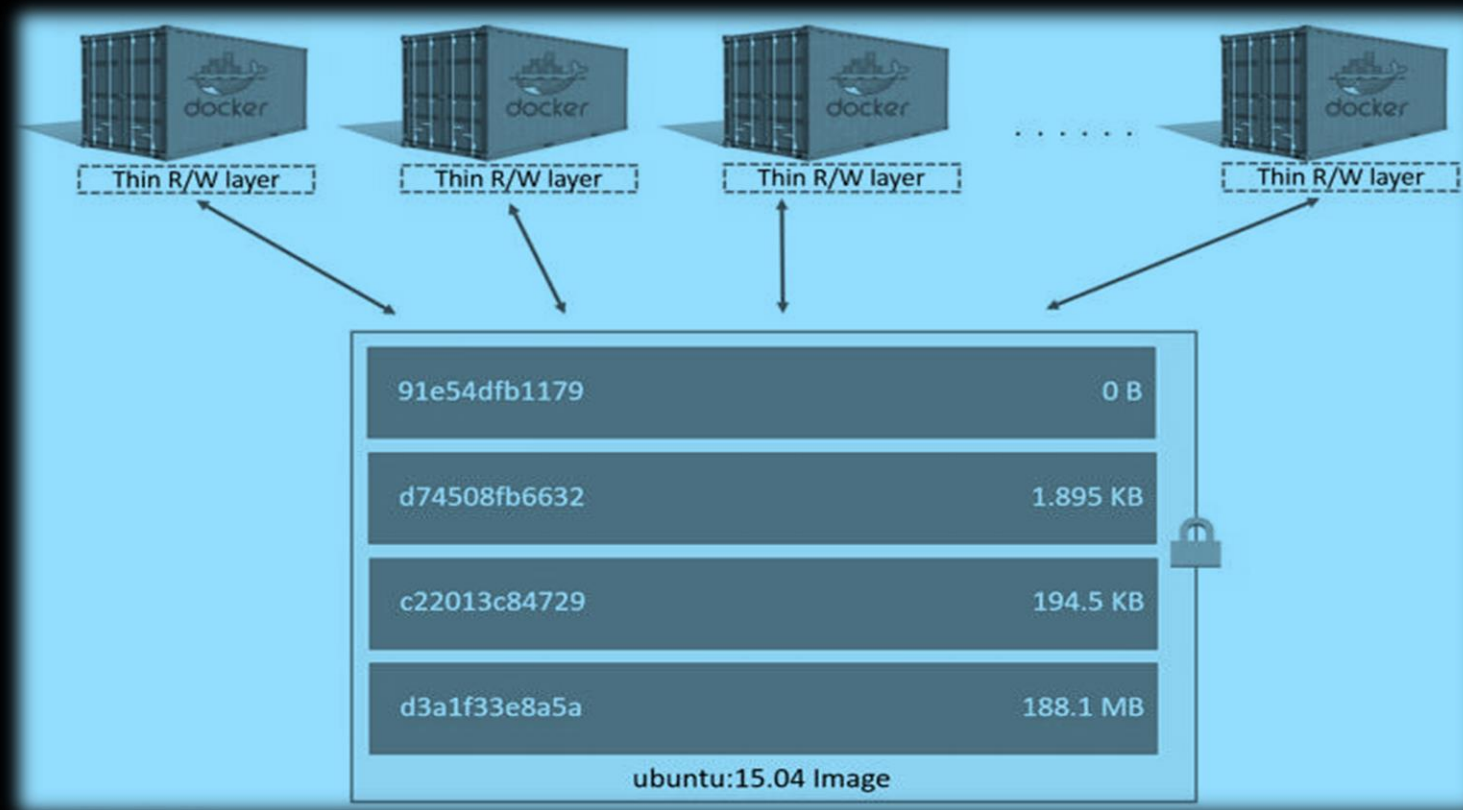
Docker Hub is the default registry

**Cloud-based Registries:** Docker Hub, Google Container Registry (GCR) and Elastic Container Registry (ECR)

**In-house/local registries:** Nexus 3+, JFrog Artifactory and Docker Trusted Registry (DTR)

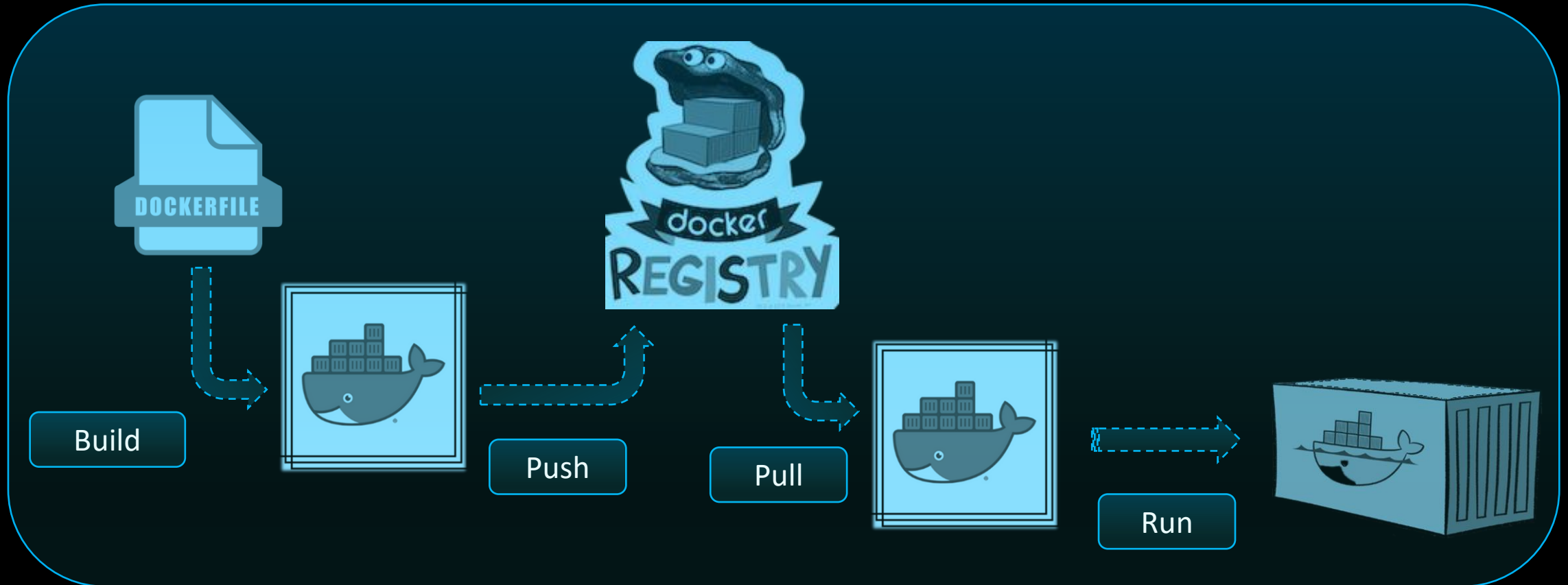
# Docker?

## ➔ Containers Run From Images



# Docker?

## ➔ Docker Container Lifecycle



# Docker?

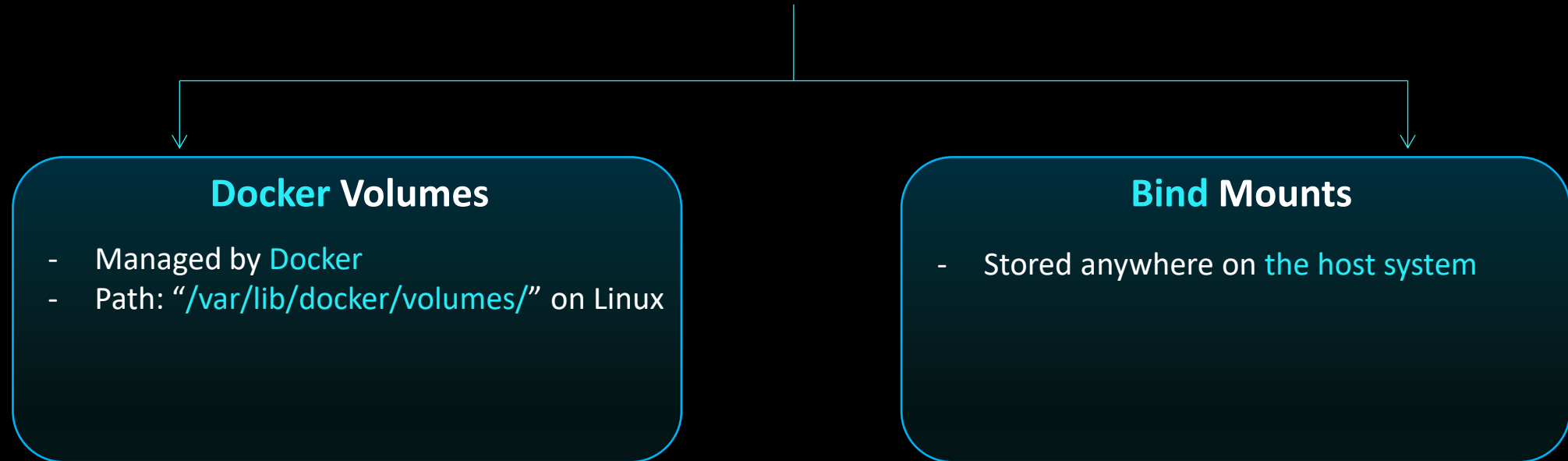
## Docker Data

-  **Data stored inside a container is not retained once the container is deleted and accessing that data externally can be difficult if another process requires it.**
-  **Because a container's writable layer is tightly bound to the underlying host, extracting or relocating that data is challenging.**

# Docker?

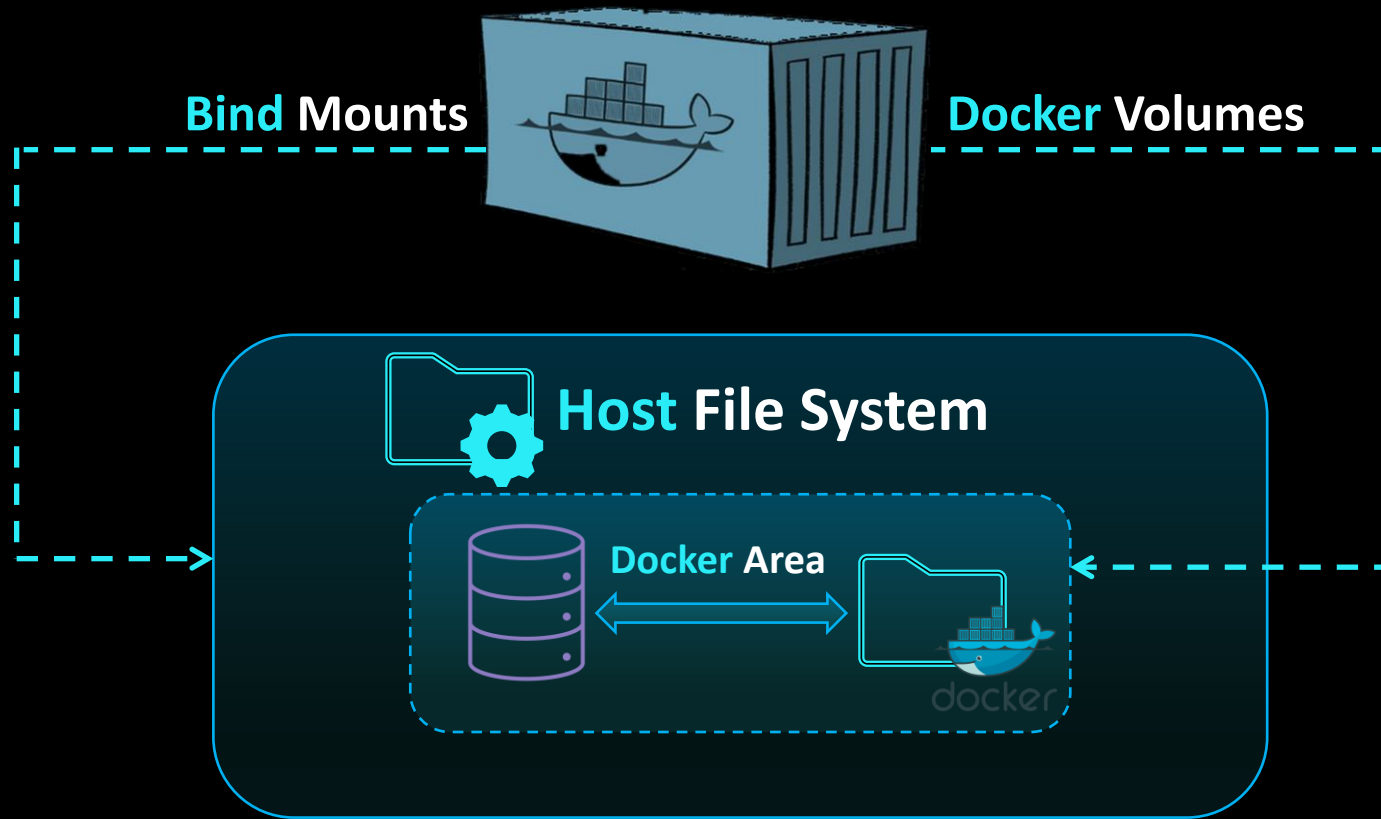
## ➡ Docker Data

☞ Docker provides two ways for containers to store files on the host machine:



# Docker?

## Docker Host



# Docker?

## ➔ Docker Volumes

☞ Docker stores volumes in a dedicated section of the host's filesystem.

on Linux: `/var/lib/docker/volumes/`

☞ This directory is managed exclusively by **Docker**, and other system processes should not alter its contents.

☞ Using **volumes** is the recommended and most reliable method for persisting data in **Docker**.

# Docker?

## **Bind Mounts**

-  **Bind mounts** can be located anywhere on the host machine's filesystem, including critical system directories.
-  They are not restricted to **Docker-managed** locations.
-  Both **Docker** containers and **non-Docker** processes on the host can change their contents at any time.

# Docker?

## ➡ **CMD vs ENTRYPOINT**

- ☞ **ENTRYPOINT** defines the main executable that always runs when the container starts.
- ☞ Commonly used when the container is meant to run a **specific application** every time.
- ☞ **ENTRYPOINT** cannot be overridden easily.

# Docker?

## ➡ **CMD vs ENTRYPOINT**

- ☞ **CMD** provides default arguments to the **ENTRYPOINT**, or acts as the command to run if no **ENTRYPOINT** exists.
- ☞ **CMD** can be overridden by passing a command at **docker run**.

# Docker?

## ➔ CMD vs ENTRYPOINT



```
FROM alpine  
CMD ["echo", "Hello CMD"]
```

```
root@andrew-ubuntu:~# docker run cmd_img  
Hello CMD
```



# Docker?

## ➔ CMD vs ENTRYPOINT



```
FROM alpine  
ENTRYPOINT ["echo"]
```

```
root@ andrew-ubuntu :~# docker run entry_img "Hello Entry"  
Hello Entry
```



# Docker?

## ➔ CMD vs ENTRYPOINT



```
FROM alpine  
ENTRYPOINT ["echo"]
```

```
root@ andrew-ubuntu :~# docker run entry_img ls  
ls (It prints 'ls')
```



# Docker?

## ➔ CMD vs ENTRYPOINT



```
FROM alpine  
ENTRYPOINT ["echo"]
```

```
root@ andrew-ubuntu :~# docker run --entrypoint ls entry_img  
➔ It lists all files
```



# Docker?

## ➡ CMD vs ENTRYPOINT



```
FROM alpine
ENTRYPOINT ["echo"]
CMD ["Hello CMD Entry"]
```

```
root@ andrew-ubuntu :~# docker run entry_cmd_img
Hello CMD Entry
```



# Docker?

## ➔ CMD vs ENTRYPOINT



```
FROM alpine  
ENTRYPOINT ["echo"]  
CMD ["Hello CMD Entry"]
```

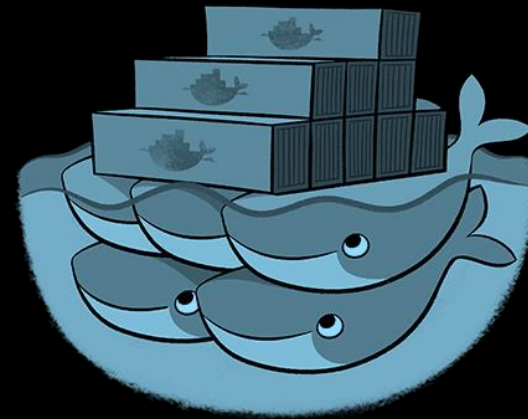
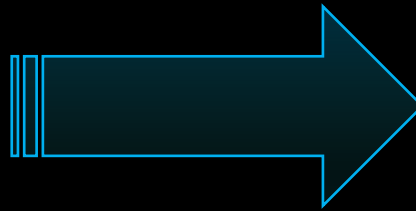
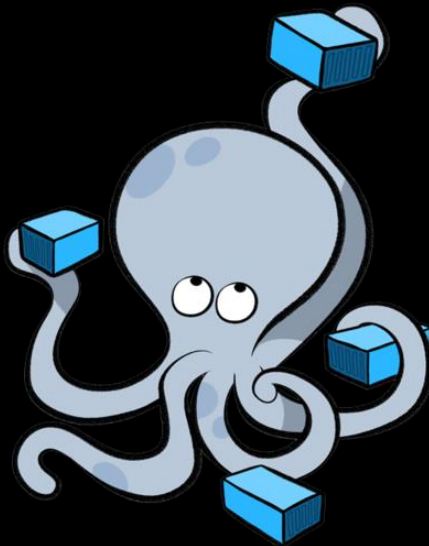
```
root@ andrew-ubuntu :~# docker run entry_cmd_img "Hello All"  
Hello All (it overrides the 'CMD' argument)
```



# Docker?

## ➔ Docker Compose

It is a tool that uses a single **YAML** file to define and run **multi-container** applications, making it easy to start, manage, scale and configure all services together using simple commands.



# Docker?

## ➔ Docker Compose

- ☞ It is a tool for defining and running **multi-container** Docker applications.
- ☞ It uses a single configuration file **compose.yml**
- ☞ It allows you to manage multiple services as **one unit**.
- ☞ It starts all services with one command **docker compose up**.

# Docker Commands ?

➡ **Open the `Lab_Commands.md`**

👉 **Download the attached `Lab_Commands.md` file.**

👉 **Open a browser and visit `Jumpshare's MD Viewer`.**

Go to: <https://jumpshare.com/viewer/md>

👉 **Upload the `.md` file**

On the `Jumpshare page`, click “`Choose MD File`” or drag & drop the `Lab_Commands.md` file.

👉 **View the rendered Markdown.**

